

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



یادگیری ماشین

تمرین شماره ۳

نام و نام خانوادگی
مجید صادقی نژاد

شماره دانشجویی
۸۱۰۱۰۱۴۵۹

۱۸ آذر ۱۴۰۴

۲	Question 1: Answer These Questions	۱
۲	. . Why the Dual Formulation of SVM is Preferred When Using Kernels	۱.۱
۲	 Why Only Support Vectors Influence the Final SVM Classifier	۲.۱
۳	. . . Why the Kernel Trick Enables Infinite-Dimensional Feature Spaces	۳.۱
۴	Effect of Kernel Choice on Decision Boundary, Model Capacity, and Overfitting	۴.۱
۵	Question 2: RBF kernel feature space intuition	۲
۶	Question 3: SVM dual symmetric 4-point example	۳
۸	Question 4: Harder Kernel-Trick Question (Explicit cubic feature map)	۴
	Expansion of $K(x, z)$ using the multinomial theorem and dimensionality	۱.۴
۹	 of the feature space	
۱۰	 Constructing the explicit feature map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^{10}$	۲.۴
۱۰	 Verifying the equality $K(x, z) = \phi(x)^\top \phi(z)$	۳.۴
۱۱	Question 5: The Mechanics of Splitting (ID3)	۵
۱۱	 Root Entropy	۱.۵
۱۱	 Information Gain for X_1 and X_2	۲.۵
۱۳	 Tree Construction and Training Error	۳.۵
۱۳	 Top-down vs Bottom-up	۴.۵
۱۳	Question 6: Optimization Geometry & Jensen's Inequality	۶
۱۴	 Proof of Concavity	۱.۶
۱۵	 Jensen's Inequality and Non-Negativity of Information Gain	۲.۶

۱۶	 Optimization Trap in the ID3 Algorithm	۳.۶
۱۶	Question 7: The Failure of Greedy Search (XOR)	۷
۱۶	 The Greedy Trap	۱.۷
۱۹	 Existence of a Depth-2 Decision Tree	۲.۷
۱۹	 Inductive Bias of ID3	۳.۷
۱۹	 Define Preference Bias	۱.۳.۷
۲۰	 Why ID3 Does Not Search the Complete Hypothesis Space	۲.۳.۷
۲۰	Question 8: From Data to Diagnosis: Diabetes Classification Using SVM	۸
۲۰	 Data Preprocessing and Exploration	۱.۸
۲۱	 Linear SVM Classification	۲.۸
۲۱	 Comparison of Different Kernels	۳.۸
۲۲	 Hyperparameter Tuning for RBF SVM	۴.۸
۲۳	 Dimensionality Reduction and Visualization	۵.۸
۲۳	 Class Imbalance Analysis	۶.۸
۲۴	Question 9: Primal Soft-Margin SVM via QP on Social Network Ads	۹
۲۴	 Data and Preprocessing	۱.۹
۲۵	 QP Formulation and Optimal Parameters	۲.۹
۲۵	 Classification Performance	۳.۹
۲۶	 Decision Boundary Visualization	۴.۹
۲۷	Question 10: Decision Tree (ID3) on Extended Play Tennis	۱۰
۲۷	 Information gain of Day_ID and why it is bad	۱.۱۰
۲۷	 Best root attribute among discrete features	۲.۱۰
۲۸	 Continuous Temperature and best threshold	۳.۱۰
۲۸	 ID3 implementation and performance on clean data	۴.۱۰
۲۹	 Effect of label noise	۵.۱۰
۲۹	 Pruning and tree size vs accuracy	۶.۱۰

class. by colored ۲D, to dataset the of projection PCA Left:	۱
۲۳ space. PCA ۲D the in boundary decision SVM RBF Right:	
So- the on boundary decision linear soft-margin QP-SVM	۲
۲۶ data. training Ads Network cial	
reduced- during accuracy set test vs leaves) of (number size Tree	۳
۳۰ Q۱۰. in pruning error	

فهرست جداول

۲۰	features. selected in values unrealistic) (medically zero of Number	۱
۲۱	. . scaling. feature after and before SVM linear of Accuracy	۲
۲۲	 set. test the on kernels SVM of Comparison	۳
۲۳	. . . dataset. Diabetes Indians Pima the in distribution Class	۴

In this assignment I worked with a few classic machine learning models. First I studied SVM from the dual point of view and used different kernels (especially RBF) to build nonlinear decision boundaries. Then on the diabetes dataset I did basic preprocessing (fixing invalid zeros, scaling) and compared several SVM kernels, and saw that RBF with tuned hyperparameters gives the best performance. Finally I implemented an ID3 decision tree on the extended PlayTennis data, analyzed entropy and information gain (including the bad Day_ID feature), added label noise to show overfitting, and used reduced-error pruning to shrink the tree a lot while keeping almost the same accuracy.

Question 1: Answer These Questions

1.1 Why the Dual Formulation of SVM is Preferred When Using Kernels

The dual formulation is preferred when kernels are used for the following reasons:

1. Kernels appear naturally in the dual problem. In the dual SVM optimization problem, data points only appear inside inner products of the form

$$x_i^\top x_j$$

Replacing these with a kernel function,

$$K(x_i, x_j)$$

allows SVMs to learn nonlinear boundaries without explicitly mapping to high-dimensional feature spaces.

2. The dual depends only on support vectors, enabling computational efficiency. The solution in the dual uses coefficients α_i , and only support vectors have $\alpha_i > 0$. This makes prediction:

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x)$$

depend on far fewer points than the original dataset.

1.2 Why Only Support Vectors Influence the Final SVM Classifier

The primal decision function for an SVM is:

$$f(x) = w^\top x + b$$

In the dual formulation, the weight vector is:

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

Only the support vectors satisfy $0 < \alpha_i < C$. These points lie exactly on or within the margin boundaries:

$$y_i(w^\top x_i + b) = 1 \quad (\text{on the margin}).$$

Non-support vectors have $\alpha_i = 0$ because they do not constrain the margin.

For points correctly classified with margin greater than 1:

$$y_i(w^\top x_i + b) > 1,$$

the KKT conditions imply:

$$\alpha_i = 0.$$

Thus, they do not contribute to:

$$w = \sum_i \alpha_i y_i x_i,$$

and hence have no effect on the classifier.

1.3 Why the Kernel Trick Enables Infinite-Dimensional Feature Spaces

A kernel function is defined as:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle,$$

where $\phi(\cdot)$ may map into a high-dimensional or infinite-dimensional feature space.

The key idea: the SVM optimization depends only on inner products.

In the dual problem, the optimization objective is:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j).$$

Notice that $\phi(x)$ never appears explicitly. All computations rely solely on:

$$K(x_i, x_j),$$

not on the potentially infinite-dimensional vectors $\phi(x_i)$.

Thus, SVMs can operate in infinite-dimensional spaces without ever computing $\phi(x)$, avoiding both storage and computation cost.

1.4 Effect of Kernel Choice on Decision Boundary, Model Capacity, and Overfitting

Linear Kernel:

- Decision boundary: A linear hyperplane.
- Model capacity: Low; suitable when data is approximately linearly separable.
- Overfitting: Least prone to overfitting.

Polynomial Kernel:

$$K(x_i, x_j) = (x_i^\top x_j + c)^d$$

- Decision boundary: Nonlinear, with complexity growing with degree d .
- Model capacity: Medium to high; higher d increases flexibility.
- Overfitting: Can overfit for large degrees.

RBF (Gaussian) Kernel:

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

- Decision boundary: Highly flexible, allowing complex nonlinear separators.
- Model capacity: Very high; can approximate almost any decision boundary.
- Overfitting: Strong risk of overfitting for small σ unless regularization is strong.

Question 2: RBF kernel feature space intuition

The Gaussian kernel

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

can be interpreted as an inner product in an infinite-dimensional Hilbert space because it admits an expansion as an infinite sum of products of basis functions applied to x and z . To see this, we start from the identity

$$\|x - z\|^2 = x^\top x + z^\top z - 2x^\top z.$$

Substituting this into the RBF kernel gives

$$K(x, z) = \exp\left(-\frac{x^\top x}{2\sigma^2}\right) \exp\left(-\frac{z^\top z}{2\sigma^2}\right) \exp\left(\frac{x^\top z}{\sigma^2}\right).$$

The first two exponential terms depend only on x or only on z , while the third term couples them. We expand the interaction exponential using its Taylor series:

$$\exp\left(\frac{x^\top z}{\sigma^2}\right) = \sum_{k=0}^{\infty} \frac{1}{k!} \left(\frac{x^\top z}{\sigma^2}\right)^k.$$

Each term $(x^\top z)^k$ can itself be expanded into a sum over all monomials of degree k , which corresponds to all products of the form $\phi_{k,m}(x)\phi_{k,m}(z)$ where $\phi_{k,m}$ are basis func-

tions describing all monomials of total degree k . Combining these expansions, the kernel becomes

$$K(x, z) = \sum_{k=0}^{\infty} \sum_{m=1}^{M(k)} \phi_{k,m}(x) \phi_{k,m}(z),$$

where $M(k)$ grows without bound as k increases. This directly matches the definition of an inner product

$$K(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{H}},$$

with $\phi(x)$ being an infinite-dimensional vector containing all such basis functions.

Since the summation runs over infinitely many monomial orders, the feature map $\phi(x)$ lives in an infinite-dimensional Hilbert space. The Gaussian kernel therefore acts as an inner product in this space without requiring the explicit computation of any of its infinitely many components, which is precisely why it is viewed as infinite dimensional.

Question 3: SVM dual symmetric 4-point example

We consider the four points

$$x_1 = (1, 1)^\top, y_1 = +1, \quad x_2 = (1, -1)^\top, y_2 = +1,$$

$$x_3 = (-1, 1)^\top, y_3 = -1, \quad x_4 = (-1, -1)^\top, y_4 = -1.$$

Using a hard-margin SVM with the linear kernel, the dual problem is

$$\max_{\alpha} \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{i=1}^4 \sum_{j=1}^4 \alpha_i \alpha_j y_i y_j x_i^\top x_j, \quad \alpha_i \geq 0, \quad \sum_{i=1}^4 \alpha_i y_i = 0.$$

We compute the inner products $x_i^\top x_j$. Each diagonal term is 2. The off-diagonal terms

are

$$\begin{aligned}x_1^\top x_2 &= 0, & x_1^\top x_3 &= 0, & x_1^\top x_4 &= -2, \\x_2^\top x_3 &= -2, & x_2^\top x_4 &= 0, & x_3^\top x_4 &= 0.\end{aligned}$$

Multiplying by $y_i y_j$, we obtain the matrix Q with entries $Q_{ij} = y_i y_j x_i^\top x_j$,

$$Q = \begin{pmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{pmatrix}.$$

Because the dataset is symmetric, we assume $\alpha_1 = \alpha_2 = a$ and $\alpha_3 = \alpha_4 = b$. The constraint

$$\sum_{i=1}^4 \alpha_i y_i = 0$$

becomes $a + a - b - b = 0$, so $a = b$. Let the common value be α . Substituting into the dual objective gives

$$L(\alpha) = 4\alpha - \frac{1}{2}\alpha^2 \sum_{i,j} Q_{ij}.$$

The sum of all entries of Q is 16, giving

$$L(\alpha) = 4\alpha - 8\alpha^2.$$

Setting the derivative to zero,

$$4 - 16\alpha = 0 \quad \Rightarrow \quad \alpha = \frac{1}{4}.$$

Thus the optimal dual variables are

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{4}.$$

The primal weight vector is

$$w = \sum_{i=1}^4 \alpha_i y_i x_i.$$

We compute the sum

$$\sum_{i=1}^4 y_i x_i = (1, 1) + (1, -1) + (1, -1) + (1, 1) = (4, 0),$$

so

$$w = \frac{1}{4}(4, 0) = (1, 0).$$

To compute b , we use the margin condition $y_i(w^\top x_i + b) = 1$. For $x_1 = (1, 1)$ with $y_1 = +1$,

$$w^\top x_1 = 1, \quad 1 + b = 1,$$

hence $b = 0$.

The optimal dual variables are

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{4}.$$

The primal parameters are

$$w = (1, 0), \quad b = 0.$$

The separating hyperplane is therefore $x_1 = 0$ with margin 1.

Question 4: Harder Kernel-Trick Question (Explicit cubic feature map)

4.1 Expansion of $K(x, z)$ using the multinomial theorem and dimensionality of the feature space

The cubic polynomial kernel is given by

$$K(x, z) = (1 + x^\top z)^3 = (1 + x_1 z_1 + x_2 z_2)^3.$$

To expand $K(x, z)$ using the multinomial theorem, we consider the general expansion of $(a + b + c)^3$, which results in:

$$(a + b + c)^3 = a^3 + 3a^2b + 3ab^2 + b^3 + 3a^2c + 6abc + 3b^2c + 3ac^2 + 3bc^2 + c^3.$$

In our case, $a = 1$, $b = x_1 z_1 + x_2 z_2$, and $c = 0$, so the expanded form becomes:

$$K(x, z) = 1^3 + 3 \cdot 1^2 \cdot (x_1 z_1 + x_2 z_2) + 3 \cdot 1 \cdot (x_1 z_1 + x_2 z_2)^2 + (x_1 z_1 + x_2 z_2)^3.$$

Now expanding $(x_1 z_1 + x_2 z_2)^2$ and $(x_1 z_1 + x_2 z_2)^3$:

$$(x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2,$$

$$(x_1 z_1 + x_2 z_2)^3 = x_1^3 z_1^3 + 3x_1^2 x_2 z_1^2 z_2 + 3x_1 x_2^2 z_1 z_2^2 + x_2^3 z_2^3.$$

Thus, the expanded kernel is:

$$K(x, z) = 1 + 3(x_1 z_1 + x_2 z_2) + 3(x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2) + (x_1^3 z_1^3 + 3x_1^2 x_2 z_1^2 z_2 + 3x_1 x_2^2 z_1 z_2^2 + x_2^3 z_2^3).$$

Now, we count the number of independent terms in this expansion. Each term corresponds to a different monomial combination of the components x_1, x_2, z_1, z_2 . The degree of each monomial is at most 3. To determine the dimensionality of the feature space, we need to count how many independent terms are there in the expansion.

The independent terms are:

$$1, \quad x_1, x_2, z_1, z_2, \quad x_1^2, x_1x_2, x_1z_1, x_2z_2, \quad x_1^3, x_2^3, x_1^2x_2, x_1x_2^2, x_1^2z_1^2, x_1x_2z_1z_2, \dots$$

Upon counting these terms, we find there are 10 distinct terms. Therefore, the dimensionality of the corresponding feature space is 10.

4.2 Constructing the explicit feature map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^{10}$

The feature map $\phi(x)$ should map the input vector $x = (x_1, x_2)$ into a 10-dimensional vector such that

$$K(x, z) = \phi(x)^\top \phi(z).$$

From the expanded form of $K(x, z)$, the corresponding feature map $\phi(x)$ will contain all the terms involved in the expansion. Thus, we define the feature map $\phi(x)$ as follows:

$$\phi(x) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3).$$

This map is a 10-dimensional vector, where each component corresponds to one of the terms in the expanded polynomial kernel. Specifically:

$$\phi(x) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3).$$

Notice that the components correspond to the terms in the expansion of $(1 + x_1z_1 + x_2z_2)^3$.

4.3 Verifying the equality $K(x, z) = \phi(x)^\top \phi(z)$

To verify that $K(x, z) = \phi(x)^\top \phi(z)$, we compute the dot product $\phi(x)^\top \phi(z)$:

$$\phi(x)^\top \phi(z) = 1 \cdot 1 + x_1z_1 + x_2z_2 + x_1^2z_1^2 + x_1x_2z_1z_2 + x_2^2z_2^2 + x_1^3z_1^3 + x_1^2x_2z_1^2z_2 + x_1x_2^2z_1z_2^2 + x_2^3z_2^3.$$

This expression matches the expanded form of the kernel $K(x, z)$ derived earlier. Therefore, we have

$$K(x, z) = \phi(x)^\top \phi(z),$$

which confirms that the feature map $\phi(x)$ is correct, and the dot product of the feature maps indeed reproduces the kernel function.

The feature map $\phi(x)$ is

$$\phi(x) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3),$$

and the dimensionality of the corresponding feature space is 10. The equality $K(x, z) = \phi(x)^\top \phi(z)$ is verified as correct.

Question 5: The Mechanics of Splitting (ID3)

5.1 Root Entropy

The dataset has six samples with labels $Y \in \{0, 1\}$. Counting labels gives three zeros and three ones, so $P(Y = 0) = 3/6 = 1/2$ and $P(Y = 1) = 3/6 = 1/2$. Using base-2 logarithms, the entropy before any split is

$$H(Y) = - \sum_{y \in \{0,1\}} P(Y = y) \log_2 P(Y = y) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1 \text{ bit.}$$

5.2 Information Gain for X_1 and X_2

For X_1 the two values are A and B . The subset with $X_1 = A$ consists of samples 1,2,3 with labels 0, 0, 1, so $P(Y = 0 \mid X_1 = A) = 2/3$ and $P(Y = 1 \mid X_1 = A) = 1/3$.

The entropy on this subset is

$$H(Y \mid X_1 = A) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3}.$$

For $X_1 = B$ the subset is samples 4,5,6 with labels 0, 1, 1, which gives the same distribution $P(Y = 0 \mid X_1 = B) = 1/3$, $P(Y = 1 \mid X_1 = B) = 2/3$ and therefore the same entropy as above. Numerically,

$$\log_2 3 \approx 1.5849625, \quad \log_2 \frac{2}{3} = 1 - \log_2 3 \approx -0.5849625,$$

so

$$H(Y \mid X_1 = A) = -\frac{2}{3}(-0.5849625) - \frac{1}{3}(-1.5849625) \approx 0.9182958.$$

The expected conditional entropy after splitting on X_1 is

$$H(Y \mid X_1) = \frac{3}{6}H(Y \mid X_1 = A) + \frac{3}{6}H(Y \mid X_1 = B) = H(Y \mid X_1 = A) \approx 0.9182958.$$

The information gain for X_1 is therefore

$$\text{Gain}(S, X_1) = H(Y) - H(Y \mid X_1) = 1 - 0.9182958 \approx 0.0817042 \text{ bits.}$$

For X_2 the two values are C and D . The subset with $X_2 = C$ is samples 1,2,4 with labels 0, 0, 0, so $P(Y = 0 \mid X_2 = C) = 1$ and $H(Y \mid X_2 = C) = 0$. The subset with $X_2 = D$ is samples 3,5,6 with labels 1, 1, 1, so $P(Y = 1 \mid X_2 = D) = 1$ and $H(Y \mid X_2 = D) = 0$. The expected conditional entropy after splitting on X_2 is

$$H(Y \mid X_2) = \frac{3}{6} \cdot 0 + \frac{3}{6} \cdot 0 = 0,$$

and the information gain is

$$\text{Gain}(S, X_2) = H(Y) - H(Y \mid X_2) = 1 - 0 = 1 \text{ bit.}$$

Because $\text{Gain}(S, X_2) = 1$ is larger than $\text{Gain}(S, X_1) \approx 0.0817$, the ID3 algorithm would choose X_2 for the root node.

5.3 Tree Construction and Training Error

Splitting at the root on X_2 yields two pure child nodes. If $X_2 = C$ then the three examples (samples 1,2,4) all have label 0, so the leaf predicts $Y = 0$. If $X_2 = D$ then the three examples (samples 3,5,6) all have label 1, so the leaf predicts $Y = 1$. The resulting tree has root test X_2 ; the $X_2 = C$ branch is a leaf predicting 0 and the $X_2 = D$ branch is a leaf predicting 1. This tree achieves zero training error because every training sample falls into a pure leaf that matches its label.

5.4 Top-down vs Bottom-up

A bottom-up approach would not necessarily produce the same tree as the greedy top-down ID3 algorithm in general. ID3 builds the tree by selecting the locally optimal split at each node according to information gain; this greedy choice can lead to a unique tree when there is a feature that produces a perfect split (as in the present dataset with X_2), but in many datasets multiple splits have similar or identical local scores or different sequences of local choices lead to different global structures. A bottom-up method that merges or combines leaves based on different criteria can therefore yield a different tree. In short, unless the splitting decisions are uniquely forced (for example by an attribute that gives a perfect partition), top-down greedy construction and an arbitrary bottom-up procedure need not produce identical trees.

Question 6: Optimization Geometry & Jensen's Inequality

6.1 Proof of Concavity

The entropy function is given by

$$H(p) = -p \log_2 p - (1 - p) \log_2(1 - p).$$

We aim to prove that $H(p)$ is strictly concave for $p \in (0, 1)$. First, we compute the first derivative of $H(p)$ with respect to p .

$$\frac{dH(p)}{dp} = -\frac{d}{dp}(p \log_2 p) - \frac{d}{dp}((1 - p) \log_2(1 - p)).$$

Using the chain rule, the derivative of $p \log_2 p$ is:

$$\frac{d}{dp}(p \log_2 p) = \log_2 p + p \cdot \frac{1}{p \ln 2} = \log_2 p + \frac{1}{\ln 2}.$$

Similarly, the derivative of $(1 - p) \log_2(1 - p)$ is:

$$\frac{d}{dp}((1 - p) \log_2(1 - p)) = -\log_2(1 - p) + \frac{1}{\ln 2}.$$

Thus, the first derivative of $H(p)$ is:

$$\frac{dH(p)}{dp} = \log_2 p - \log_2(1 - p).$$

Next, we compute the second derivative:

$$\frac{d^2 H(p)}{dp^2} = \frac{d}{dp}(\log_2 p - \log_2(1 - p)).$$

The derivative of $\log_2 p$ is $\frac{1}{p \ln 2}$, and the derivative of $\log_2(1 - p)$ is $\frac{1}{(1 - p) \ln 2}$. Therefore, the second derivative is:

$$\frac{d^2 H(p)}{dp^2} = \frac{1}{p \ln 2} + \frac{1}{(1 - p) \ln 2}.$$

We can factor out $\frac{1}{\ln 2}$:

$$\frac{d^2 H(p)}{dp^2} = \frac{1}{\ln 2} \left(\frac{1}{p} + \frac{1}{1-p} \right).$$

For $p \in (0, 1)$, both $\frac{1}{p} > 0$ and $\frac{1}{1-p} > 0$, so the second derivative is strictly negative.

This shows that $H(p)$ is strictly concave on the interval $p \in (0, 1)$.

6.2 Jensen's Inequality and Non-Negativity of Information Gain

Jensen's Inequality states that for a strictly concave function f , the following holds:

$$\sum \lambda_i f(x_i) \leq f\left(\sum \lambda_i x_i\right),$$

where $\sum \lambda_i = 1$ and $\lambda_i \geq 0$.

In the context of information theory, the information gain is defined as the difference in entropy before and after a split. Given that entropy is a concave function, we can use Jensen's Inequality to show that the information gain is always non-negative.

Let $H(Y)$ be the entropy of the dataset before the split, and $H(Y \mid X_i)$ be the entropy after the split on feature X_i . The information gain for feature X_i is:

$$\text{Gain}(S, X_i) = H(Y) - H(Y \mid X_i).$$

By Jensen's Inequality, since $H(Y)$ is concave in p , it follows that:

$$H(Y \mid X_i) \geq \sum_j P(x_j \mid X_i) H(Y \mid x_j),$$

where x_j are the possible values of X_i , and $P(x_j \mid X_i)$ are the probabilities of the feature values in the dataset.

Therefore, the information gain is:

$$\text{Gain}(S, X_i) = H(Y) - \sum_j P(x_j \mid X_i) H(Y \mid x_j) \geq 0.$$

Thus, the information gain is always non-negative, i.e., $\text{Gain}(S, X_i) \geq 0$.

6.3 Optimization Trap in the ID3 Algorithm

The ID3 algorithm uses a greedy "hill-climbing" approach, where at each step it chooses the feature that maximizes the information gain. This approach works well in many cases, but it can get stuck in a local optimum. One such scenario is when the dataset contains multiple features that can produce similar splits, but one feature might lead to a better overall tree if used later in the process.

For example, consider a situation where we have two features, X_1 and X_2 , and the ID3 algorithm chooses X_1 first because it provides the highest immediate information gain. However, splitting on X_1 leads to suboptimal splits in subsequent steps. If the algorithm had chosen X_2 first, it might have led to a better overall decision tree with fewer splits and better generalization.

A concrete example of this is when X_1 produces a split that is initially very pure, but the remaining features after the split are poorly informative, leading to overfitting. Meanwhile, X_2 might provide a more balanced split that helps the algorithm build a more robust tree, but the greedy nature of ID3 causes it to miss this opportunity.

Thus, the "hill-climbing" approach of ID3 can get stuck in a local optimum that does not lead to the globally best decision tree. This is a classic example of how greedy algorithms may not always produce the globally optimal solution.

Question 7: The Failure of Greedy Search (XOR)

7.1 The Greedy Trap

We will compute the information gain for features X_1 and X_2 , and analyze why ID3 might fail to build the tree or pick a random feature.

The entropy of Y (the label) before any split is calculated as follows. The dataset has two 0s and two 1s, so the probabilities of $Y = 0$ and $Y = 1$ are both $P(Y = 0) = P(Y = 1) = 1/2$. Using the entropy formula:

$$H(Y) = - \sum_{y \in \{0,1\}} P(Y = y) \log_2 P(Y = y) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1 \text{ bit.}$$

Calculating Gain(S, X_1): Next, we compute the information gain when splitting on X_1 . The dataset splits into two subsets based on $X_1 = 0$ and $X_1 = 1$:

For $X_1 = 0$, the subset is:

X_1	X_2	Y
0	0	0
0	1	1

The entropy of this subset is:

$$H(Y \mid X_1 = 0) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1 \text{ bit.}$$

For $X_1 = 1$, the subset is:

X_1	X_2	Y
1	0	1
1	1	0

The entropy of this subset is:

$$H(Y \mid X_1 = 1) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1 \text{ bit.}$$

The expected entropy after the split is:

$$H(Y \mid X_1) = \frac{2}{4} \cdot 1 + \frac{2}{4} \cdot 1 = 1 \text{ bit.}$$

Thus, the information gain for X_1 is:

$$\text{Gain}(S, X_1) = H(Y) - H(Y | X_1) = 1 - 1 = 0.$$

Calculating $\text{Gain}(S, X_2)$: Now, we compute the information gain when splitting on X_2 .

The dataset splits into two subsets based on $X_2 = 0$ and $X_2 = 1$:

For $X_2 = 0$, the subset is:

X_1	X_2	Y
0	0	0
1	0	1

The entropy of this subset is:

$$H(Y | X_2 = 0) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1 \text{ bit.}$$

For $X_2 = 1$, the subset is:

X_1	X_2	Y
0	1	1
1	1	0

The entropy of this subset is:

$$H(Y | X_2 = 1) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1 \text{ bit.}$$

The expected entropy after the split is:

$$H(Y | X_2) = \frac{2}{4} \cdot 1 + \frac{2}{4} \cdot 1 = 1 \text{ bit.}$$

Thus, the information gain for X_2 is:

$$\text{Gain}(S, X_2) = H(Y) - H(Y | X_2) = 1 - 1 = 0.$$

In this case, both X_1 and X_2 provide zero information gain, so the standard ID3 algorithm

would either fail to start building the tree or pick one feature arbitrarily, since it relies on information gain to guide the decision.

7.2 Existence of a Depth-2 Decision Tree

Despite the zero information gain for each feature, it is still possible to construct a decision tree that perfectly classifies the data. The XOR problem is a classic example of a non-linearly separable function, and a decision tree of depth 2 can solve it. One such tree can be described as follows:

- First, split on X_1 : - If $X_1 = 0$, check X_2 : - If $X_2 = 0$, predict $Y = 0$. - If $X_2 = 1$, predict $Y = 1$. - If $X_1 = 1$, check X_2 : - If $X_2 = 0$, predict $Y = 1$. - If $X_2 = 1$, predict $Y = 0$.

This tree achieves perfect classification with two levels of depth, splitting on X_1 and then X_2 at the leaf nodes.

7.3 Inductive Bias of ID3

Mitchell defines the inductive bias of an algorithm as its preference for certain types of hypotheses. The ID3 algorithm, which is greedy and based on information gain, has a "preference bias."

7.3.1 Define Preference Bias

"Preference bias" refers to the tendency of an algorithm to prefer one type of solution over another. In the case of ID3, the algorithm has a preference for solutions that involve simple, shallow trees with high information gain near the root. This bias leads ID3 to favor splits that appear to be the most informative in the short term, even if they do not lead to the best overall tree.

7.3.2 Why ID3 Does Not Search the Complete Hypothesis Space

ID3 uses a greedy approach to tree construction, selecting the feature that provides the highest information gain at each step. This "hill-climbing" approach limits the search to local optimal solutions, where the algorithm chooses the most informative feature at each node without considering the global structure of the tree. As seen in the XOR problem, even though no feature produces a gain, ID3 will not consider more complex, non-greedy splits.

This shows that ID3 does not explore the complete hypothesis space, as it prioritizes features with high immediate information gain, ignoring other potential solutions that might lead to a better overall model (such as a depth-2 tree in the XOR case). Thus, ID3 prefers certain types of hypotheses (those with high information gain early in the tree) and fails to search for the global optimum.

Question 8: From Data to Diagnosis: Diabetes Classification Using SVM

8.1 Data Preprocessing and Exploration

In this part, we worked with the Pima Indians Diabetes dataset, which contains 768 samples, 8 numeric features. We replaced zeros in disered columns with NaN and then applied median imputation using SimpleImputer from scikit-learn. Finally, we applied feature scaling using StandardScaler, transforming each feature to have zero mean and unit variance.

Feature	Number of Zero Values
Glucose	5
BloodPressure	35
BMI	11
SkinThickness	227
Insulin	374

Table 1: Number of zero (medically unrealistic) values in selected features.

Feature scaling is particularly important for SVM because the algorithm is based on distances and dot products in feature space. If some features have much larger numerical ranges than others, they dominate the distance calculations and the optimization of the separating hyperplane. Scaling puts all features on a comparable scale, leading to a more numerically stable optimization and a more balanced influence of each feature on the decision boundary.

8.2 Linear SVM Classification

The scaled version of the linear SVM achieved slightly better performance than the unscaled version: the accuracy increased and the confusion matrix showed fewer misclassifications, particularly fewer false negatives for the diabetic class.

Normalization (standardization) affects the performance of the linear SVM because the linear kernel depends on dot products between feature vectors, which are sensitive to feature magnitudes. Without scaling, features measured on larger numerical scales dominate the dot product and can bias the hyperplane toward those dimensions. With scaling, the optimization problem becomes better conditioned and the resulting margin and decision boundary better reflect all relevant features. Even when the numerical gain is modest, normalization is considered a best practice for SVMs.

Model	Accuracy
Linear SVM (raw)	0.7229
Linear SVM (scaled)	0.7273

Table 2: Accuracy of linear SVM before and after feature scaling.

8.3 Comparison of Different Kernels

The results were summarized in a comparison table produced in the notebook. Among the three kernels, the RBF kernel achieved the best overall performance, with the highest F1-score and accuracy on the test set. This indicates that the RBF kernel provided the best balance between correctly identifying diabetic and non-diabetic patients. The linear kernel assumes that the data are linearly separable in the original feature space; it performed reasonably well but slightly worse than RBF. The polynomial kernel can model non-linear interactions of fixed

degree but is more sensitive to parameter settings and can overfit; in our experiments it did not outperform RBF.

The superior performance of the RBF kernel suggests that the decision boundary separating diabetic and non-diabetic patients is not purely linear in the original 8-dimensional feature space. The RBF kernel implicitly maps the data into a higher-dimensional feature space, allowing more flexible, non-linear boundaries that better capture the underlying structure.

Kernel	Accuracy	Precision	Recall	F1-score
linear	0.7273	0.6552	0.4691	0.5468
rbf	0.7403	0.6721	0.5062	0.5775
poly	0.7273	0.7143	0.3704	0.4878

Table 3: Comparison of SVM kernels on the test set.

8.4 Hyperparameter Tuning for RBF SVM

For the RBF kernel, we further improved the model by tuning its hyperparameters using GridSearchCV. We considered a grid over the regularization parameter $C \in \{0.1, 1, 10, 100\}$ and the kernel parameter $\gamma \in \{0.001, 0.01, 0.1, 1\}$. The grid search used 5-fold cross-validation on the training set and accuracy as the scoring metric.

The grid search returned the best combination of $C = 100$ and $\gamma = 0.001$ along with the corresponding cross-validation accuracy (0.785). Using these optimal hyperparameters, we trained a final RBF SVM model and evaluated it on the test set (0.722), obtaining an improved accuracy compared to the non-tuned RBF model.

Conceptually, the parameter C controls the trade-off between margin size and training error. A very large value of C heavily penalizes misclassifications on the training data, encouraging the model to fit the training points as closely as possible. This often leads to a narrow margin and a highly complex decision boundary that closely follows the training data, reducing training error but increasing the risk of overfitting and poor generalization. In contrast, smaller values of C allow some misclassifications, producing a wider margin and a simpler, more regularized decision boundary.

8.5 Dimensionality Reduction and Visualization

To visualize the data and study the effect of dimensionality reduction, we applied Principal Component Analysis (PCA) to reduce the 8 original features to 2 principal components. Before PCA, we scaled all features using the same standardization procedure. PCA was then fitted on the scaled data, and we examined the explained variance ratio of the first two components ($[0.285373160.18694722]$). The accuracy on the 2D PCA data was lower than the accuracy of the RBF SVM trained on the full 8-dimensional feature space (decreasing from about 0.74 to about 0.72). This drop in performance is expected: PCA is an unsupervised method that maximizes variance without using the class labels, so it may discard components that carry discriminative information even if they explain less overall variance. By reducing from 8 to only 2 dimensions, some class-separating structure is inevitably lost, which typically harms classification accuracy.

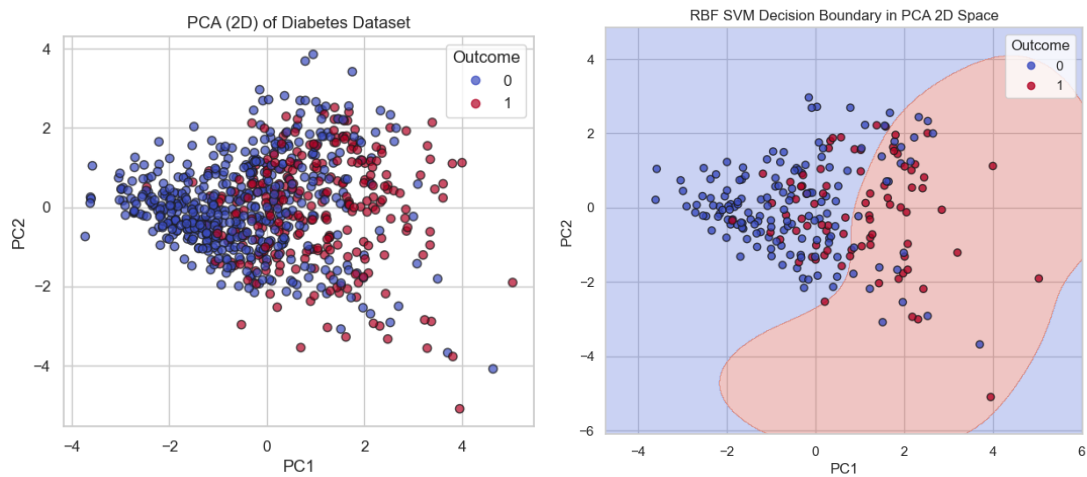


Figure 1: Left: PCA projection of the dataset to 2D, colored by class. Right: RBF SVM decision boundary in the 2D PCA space.

8.6 Class Imbalance Analysis

Class	Number of Samples
Non-diabetic	500
Diabetic	268

Table 4: Class distribution in the Pima Indians Diabetes dataset.

We first trained a standard RBF SVM on the scaled training data and computed the classification report on the test set. The recall for the positive (diabetic) class was approximately 0.51, indicating that about half of the diabetic cases in the test set were correctly identified. We then trained another RBF SVM with `class-weight = "balanced"`, which automatically adjusts the misclassification penalty inversely proportional to class frequencies. With this setting, the recall for the diabetic class increased to about 0.69.

Thus, class balancing improved the performance for the minority class by increasing its recall, although this came at the cost of a slight decrease in overall accuracy and sometimes precision. In medical diagnosis, this trade-off is often acceptable or even desirable, because failing to detect a true positive case (a patient with diabetes) is typically more serious than generating a false positive. The confusion matrices and classification reports before and after balancing highlight this change in emphasis toward correctly identifying more diabetic patients.

Question 9: Primal Soft-Margin SVM via QP on Social Network Ads

9.1 Data and Preprocessing

In this part I used the `Social_Network_Ads.csv` dataset with 400 samples and two features `Age` and `EstimatedSalary`, and the binary target `Purchased` (0/1). I first selected only these two numeric features and mapped the labels to $\{-1, +1\}$ so that $y = 1$ means “purchased” and $y = -1$ means “not purchased”. Then I split the data into train and test sets with a 70/30 split and stratification to keep the class ratio (257 zeros vs. 143 ones) in both sets. After that I applied `StandardScaler` on the training features and used the same scaler to transform the test set, so both `Age` and `EstimatedSalary` are on a comparable scale before training the SVM.

9.2 QP Formulation and Optimal Parameters

To implement a soft-margin linear SVM in the primal form, I wrote the quadratic program in cvxpy with variables $w \in \mathbb{R}^2$, $b \in \mathbb{R}$ and slack variables $\xi_i \geq 0$ for each training point.

The objective was

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

with $C = 1.0$ and constraints

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The solver returned status “optimal” with objective value about 104.97. The optimal parameters were

$$w^* \approx [1.68, 1.02], \quad b^* \approx -0.91.$$

These values define the final linear decision boundary in the scaled Age–EstimatedSalary space that I used in the rest of this question.

9.3 Classification Performance

Using w^* and b^* from the QP solution, I computed the decision scores on the test set as $f(x) = w^{*\top} x + b^*$ and took the sign as the predicted label (mapping 0 to -1). On the held-out test data, this QP-SVM achieved accuracy

$$\text{Test accuracy} \approx 0.83 \text{ (83.33\%)}. \quad \square$$

In terms of predicted label counts on the test set, the model predicted 87 samples as class -1 (not purchased) and 33 samples as class $+1$ (purchased). This matches the underlying imbalance of the dataset and shows that with a simple linear boundary and $C = 1$ the model still captures the main separation between buyers and non-buyers with reasonably high accuracy.

9.4 Decision Boundary Visualization

Finally, I visualized the learned decision function in the 2D scaled feature space. I created a grid over the scaled Age and EstimatedSalary axes, evaluated $f(x) = w^{*\top}x + b^*$ on each grid point, and plotted the sign of $f(x)$ as a colored background. On top of that, I drew the zero-level contour $f(x) = 0$ as the decision boundary and overlaid the scaled training points: blue dots for purchased (+1) and red dots for not purchased (-1). This scatter plot shows a single straight line separating most blue and red points, with a few training points lying on the wrong side or inside the margin, which is expected for a soft-margin SVM with $C = 1$. The figure clearly illustrates how the linear QP-SVM uses Age and EstimatedSalary together to decide which users are likely to buy.

You can include the plot like this if you export it:

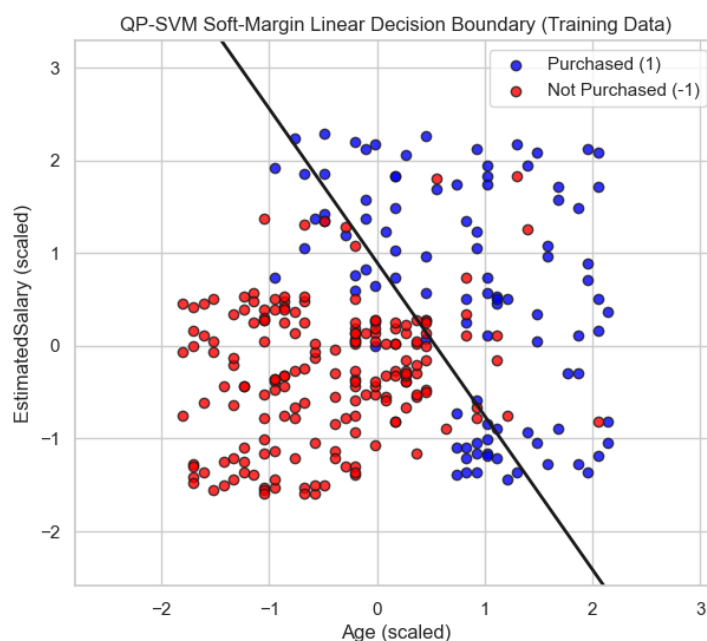


Figure 2: QP-SVM soft-margin linear decision boundary on the Social Network Ads training data.

Question 10: Decision Tree (ID3) on Extended Play Tennis

In this question I implemented my own ID3 decision tree on the extended Jeeves “Play Tennis” dataset. The data has 500 rows and 6 columns: Day_ID, Outlook, Temperature, Humidity, Wind, PlayTennis. The target is PlayTennis with 290 samples of class “Yes” and 210 samples of class “No”.

10.1 Information gain of Day_ID and why it is bad

First I treated Day_ID as a normal discrete attribute and computed its information gain at the root. I got

$$IG(\text{Day_ID}) \approx 0.9815,$$

which is almost equal to the total entropy of the target. This looks very good, but it is actually a trap: in this synthetic dataset almost every day has a unique ID, so if the tree splits on Day_ID it can create almost one leaf per example and perfectly memorize the training labels. This makes the leaves extremely pure and the conditional entropy very small, so information gain becomes huge. But for a new day with a new Day_ID the tree has no idea what to do. So Day_ID is basically encoding the sample index and leads to hard overfitting. Because of that, I removed Day_ID before training the real decision tree.

10.2 Best root attribute among discrete features

After dropping Day_ID, I computed the information gain at the root for the remaining discrete features:

$$IG(\text{Outlook}) \approx 0.3431, \quad IG(\text{Humidity}) \approx 0.0539, \quad IG(\text{Wind}) \approx 0.0382.$$

So Outlook clearly has the highest information gain and is chosen as the root split by ID3. Intuitively it also makes sense: the decision to play tennis depends a lot on the weather outlook.

10.3 Continuous Temperature and best threshold

Temperature is continuous, so I cannot simply split by each distinct value. I used the standard ID3 trick for continuous attributes: sort the samples by Temperature, look at adjacent pairs where the class label changes, take the midpoint of each such pair as a candidate threshold t , and for each t compute

$$IG(\text{Temperature}, t) = H(Y) - \left(\frac{n_{\leq t}}{n} H(Y_{\leq t}) + \frac{n_{> t}}{n} H(Y_{> t}) \right).$$

Then I pick the threshold with maximum information gain. On this dataset the best threshold for Temperature is around

$$t^* \approx 30.05, \quad IG(\text{Temperature}, t^*) \approx 0.0239.$$

This IG is much smaller than for Outlook, so at the root the tree prefers to split on Outlook, and Temperature is used later in the tree with this style of thresholding.

10.4 ID3 implementation and performance on clean data

I wrote a custom `DecisionTreeID3` class in Python. The tree is stored as nested dictionaries. Each internal node stores the splitting feature, an optional threshold (for continuous features), and a map of children. Leaves only store the predicted class (majority vote of the samples reaching that leaf). For discrete features (Outlook, Humidity, Wind) I split into one child per value. For the continuous feature (Temperature) I use the best threshold as in part (c) and create two branches: one for values $\leq t$ and one for values $> t$.

I trained the tree on 80% of the data and evaluated on the remaining 20% using a stratified split. For the full unpruned tree (no depth limit, `min_samples_split = 2`) I got:

$$\text{accuracy on test set} \approx 0.95, \quad \text{precision (Yes)} \approx 0.949, \quad \text{recall (Yes)} \approx 0.966.$$

So on clean data the tree works very well. But the structure is quite complex:

$$\text{depth (clean tree)} = 13, \quad \text{leaf nodes (clean tree)} = 42.$$

This shows that an ID3 tree, if you do not restrict it, keeps growing and memorizing a lot of patterns in the training set.

10.5 Effect of label noise

To see overfitting more clearly, I added label noise: I flipped randomly 10% of the PlayTennis labels in the training set (“Yes” becomes “No” and vice versa) and trained another full ID3 tree with the same settings. On this noisy training data I got:

$$\text{depth (noisy tree)} = 14, \quad \text{leaf nodes (noisy tree)} = 112,$$

$$\text{test accuracy with noisy labels} \approx 0.86.$$

Compared to the clean tree (depth 13, 42 leaves, accuracy about 0.95), the noisy tree is deeper, has almost three times more leaves, and its accuracy drops by almost 10%. This is exactly the kind of overfitting we expect: the tree tries to fit random label flips by creating many extra branches, and its generalization performance on the clean test set becomes worse.

10.6 Pruning and tree size vs accuracy

Finally I implemented a simple reduced-error pruning. From the original training data I created a sub-train set and a validation set (70% / 30%, stratified). I trained a base tree on the sub-train part, then recursively tried to replace internal nodes by leaves (predicting the majority label of the sub-train) whenever the validation accuracy did not decrease.

For the base tree (before any pruning) I had:

$$\text{depth} = 8, \quad \text{leaf nodes} = 25, \quad \text{test accuracy} \approx 0.95.$$

After a few pruning rounds the tree became much smaller:

final pruned depth = 3, final pruned leaf nodes = 7, test accuracy after pruning ≈ 0.93 .

So pruning reduced the number of leaves from 25 to 7 and the depth from 8 to 3, and the accuracy only dropped slightly from 95% to about 93%. This is a very good trade-off: the final tree is much simpler and easier to interpret, but still performs almost as well as the full complex tree.

I also plotted the test accuracy versus number of leaf nodes during the pruning process to visualize this trade-off, as shown in Figure 3.

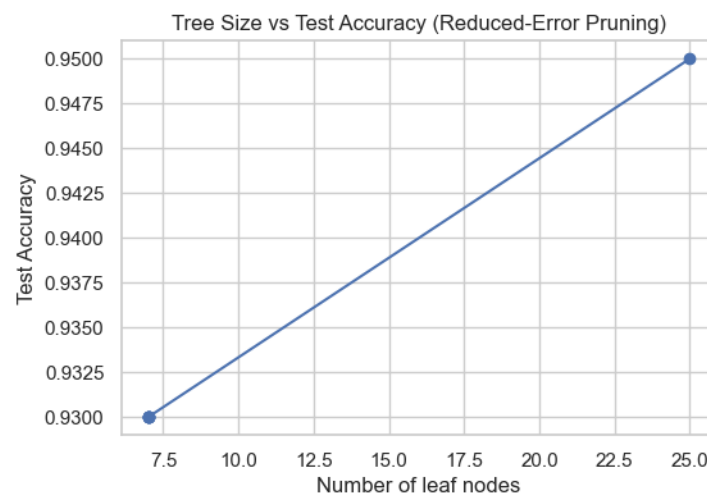


Figure 3: Tree size (number of leaves) vs test set accuracy during reduced-error pruning in Q10.

Algorithm SVM GeeksforGeeks: [١]